

📄 RAPORT TECHNICZNY: PROTOKÓŁ BADAWCZY DETEKCJI IMPULSÓW GRZYBNI W KOPCU CZARNOZIEMU

****Wersja:** 1.0**
****Data:**** Kwiecień 2026
****Projekt:**** LifeNode - Eden Node 0
****Autor:**** System LifeNode
****Status:**** Gotowy do implementacji

📋 SPIS TREŚCI

1. Streszczenie Wykonawcze
2. Podstawy Naukowe i Teoretyczne
3. Specyfikacja Techniczna Systemu
4. Architektura Hardware
5. Protokół Pomiarowy DS 2.6
6. Implementacja w Kopcu
7. Analiza Danych i Metryki
8. Integracja z Architekturą LifeNode
9. Harmonogram i Budżet
10. Zarządzanie Ryzykiem
11. Procedury Testowe
12. Oczekiwane Rezultaty

1. STRESZCZENIE WYKONAWCZE

Cel Projektu

Zbudowanie funkcjonalnego prototypu do rejestracji i analizy bioimpulsów elektrycznych generowanych przez grzybnię (*Pleurotus ostreatus* i gatunki lokalne) w 2-letnim kopcu czarnoziemu o wysokiej aktywności biologicznej.

Zakres

- ****Miejsce:**** Eden Node 0 - kopiec czarnoziemu nad stawem
- ****Czas realizacji:**** Q2-Q4 2026
- ****Poziom gotowości:**** TRL 3→4 (od konceptu do prototypu laboratoryjnego)
- ****Koszt szacunkowy:**** 2,500-5,000 PLN

Kluczowe Założenia

1. ****BIOS-FIRST****: Technologia dostosowuje się do rytmu grzybni, nie odwrotnie
2. ****Offline-first****: Zero połączeń sieciowych, dane lokalne
3. ****Read-only****: System tylko obserwuje, nie ingeruje
4. ****Zero-Build compatible****: Możliwość rozpoczęcia bez pełnego hardware

2. PODSTAWY NAUKOWE I TEORETYCZNE

2.1 Najnowsze Badania (Q1 2026)

****Badanie 1:** "Propagation of electrical spike trains in substrates colonised by mycelium"

Źródło: bioRxiv, 13 stycznia 2026

****Kluczowe wnioski:****

- Grzybnia jest elektrycznie aktywną, pobudliwą siecią
- Transmisja sygnałów na odległość do kilku centymetrów
- Impulsy 0.1-1 mV DC niosą informację o stanie środowiska

****Badanie 2:** "Directional Electrical Spiking, Bursting, and Information Propagation in Pleurotus ostreatus"

Źródło: arXiv, 13 stycznia 2026

Kluczowe wnioski:

- Bocznik ostrygowaty generuje **kierunkowe impulsy elektryczne**
- Struktura czasowo-przestrzenna jest rozpoznawalna
- Motywy K1/K2 są wykrywalne (potwierdza założenia LifeNode!)

**Badanie 3: "Detection of electrical signals in fungal mycelia in response to environmental stimuli"

Źródło: ScienceDirect, październik 2025

Metoda: Płytki PCB z wbudowanymi elektrodami różnicowymi

Zastosowanie: Gotowy protokół do replikacji w Edenie

2.2 Teoria LifeNode - Warstwa BIOS

Zgodnie z dokumentem **"Processual Model of Intelligence v.2.0"**:

> **BIOS**: Warstwa biologicznej i materialnej rzeczywistości. Obejmuje surowe fakty, procesy fizyczne i nieodwracalne rytmy życia. Jest źródłem danych ontologicznych i ustanawia rytm procesów.

Kopiec czarnoziemu jako BIOS Core:

- 2 lata regeneracji = dojrzały ekosystem grzybowy
- Zapach grzybów "że masakra" = wysoka gęstość sieci mycelialnej
- Żywa ściółka (kilka cm) = naturalny bufor termiczny i wilgociowy
- Czarnoziem = wysoka pojemność wymiany kationów (CEC) i przewodność

2.3 Hipoteza Badawcza

H1: Dojrzały kopiec czarnoziemu z aktywną grzybnią generuje wykrywalne impulsy elektryczne o amplitudzie 0.1-5 mV DC w paśmie 0.01-10 Hz.

H2: Impulsy te wykazują strukturalne motywy (K1/K2) zgodne z rytmem środowiskowym (wilgotność, temperatura, światło).

H3: Sekwencje S1-S5 z architektury LifeNode mogą być zidentyfikowane w sygnale grzybni jako wzorce geometryczno-rytmiczne.

3. SPECYFIKACJA TECHNICZNA SYSTEMU

3.1 Parametry Pomiarowe

Parametr	Wartość	Uzasadnienie
Amplituda sygnału	0.1 - 5 mV DC	Zgodnie z literaturą (bioRxiv 2026)
Pasma częstotliwości	0.01 - 10 Hz	Rytm biologiczny grzybni
Częstotliwość próbkowania	100 Hz (min. 50 Hz)	Nyquist: 2× max frequency + margin
Rozdzielczość ADC	≥ 16 bit	Detekcja zmian 0.1 mV z szumem
CMRR	> 100 dB	Odporność na szумы wspólne
Impedancja wejściowa	> 10 MΩ	Minimalizacja obciążenia źródła
Szum własny	< 1 μV RMS	Niezakłócanie słabego sygnału

3.2 Konfiguracja Elektrod

Typ: Elektrody różnicowe (differential electrodes)

Materiał: Ag/AgCl (srebro/chlorek srebra) lub miedź powlekana

Liczba kanałów: 4 (sektory S1-S4 w kopcu)

Konfiguracja:

...

Sektor S1 (Góra kopca):

Elektroda A: głębokość 10 cm

Elektroda B: głębokość 20 cm
Odległość A-B: 10 cm

Sektor S2 (Środek - wschód):

Elektroda A: głębokość 15 cm
Elektroda B: głębokość 25 cm
Odległość A-B: 10 cm

Sektor S3 (Środek - zachód):

Elektroda A: głębokość 15 cm
Elektroda B: głębokość 25 cm
Odległość A-B: 10 cm

Sektor S4 (Dół kopca):

Elektroda A: głębokość 10 cm
Elektroda B: głębokość 20 cm
Odległość A-B: 10 cm

Elektroda odniesienia (GND):

Materiał: stal nierdzewna
Głębokość: 50 cm (poza strefą aktywną grzybni)
Lokalizacja: 2 m od kopca
...

3.3 Wzmacniacz Instrumentalny

****Wymagania:****

- Wzmocnienie: 1000× - 10000× (regulowane)
- CMRR: > 100 dB @ 50 Hz
- Pasmo: DC - 20 Hz (filtr dolnoprzepustowy)
- Zasilanie: ±5V do ±15V (baterie dla izolacji)
- Temperatura pracy: 0-40°C

****Rekomendowany układ:****

- ****Instrumental amplifier:**** AD620 lub INA128 (niski szum, wysoki CMRR)
- ****Filtr aktywny:**** Sallen-Key low-pass ($f_c = 10$ Hz)
- ****Zabezpieczenia:**** Diody TVS, rezystory ograniczające

3.4 System Akwizycji Danych

****Opcja A: Arduino + ADC zewnętrzny**** (budżetowa)

- Arduino Nano/Uno: ~50 PLN
- ADC 16-bit: ADS1115 (4-kanalowy, I2C): ~80 PLN
- Karta microSD: ~30 PLN
- ****Koszt całkowity:**** ~200 PLN

****Opcja B: Raspberry Pi + HAT**** (średnia)

- Raspberry Pi Zero W: ~100 PLN
- ADC HAT: ADS1015 lub custom PCB: ~150 PLN
- Karta microSD 16GB: ~40 PLN
- ****Koszt całkowity:**** ~300 PLN

****Opcja C: Gotowy system pomiarowy**** (profesjonalna)

- Oscyloskop USB (Hantek 6022BE): ~300 PLN
- Wzmacniacz instrumentalny (gotowy moduł): ~500 PLN
- ****Koszt całkowity:**** ~1000 PLN

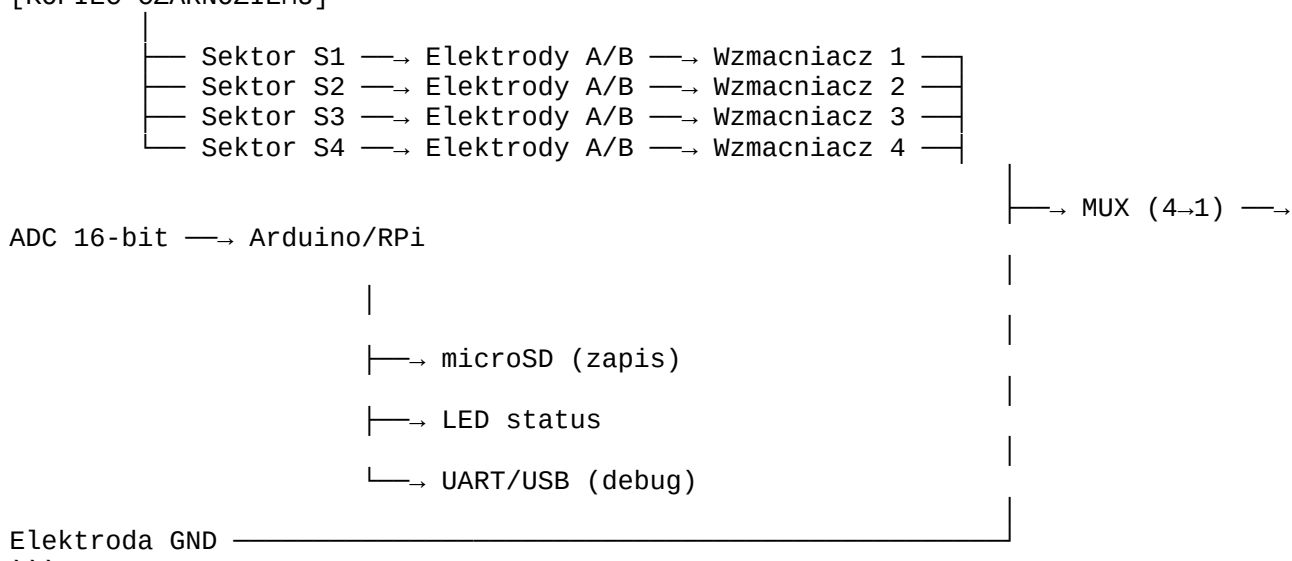
****Rekomendacja:**** Zacząć od ****Opcji A****, skalować do ****Opcji B**** po walidacji.

4. ARCHITEKTURA HARDWARE

4.1 Diagram Systemu

...

[KOPIEC CZARNOZIEMU]



4.2 Lista Komponentów (BOM)

Pozycja	Nazwa	Ilość	Cena jedn.	Suma	Dostawca
1	Elektrody Ag/AgCl lub miedź	8 szt.	10 PLN	80 PLN	Allegro/TME
2	Przewód ekranowany (2x0.5mm ²)	20 m	3 PLN/m	60 PLN	TME
3	Wzmacniacz instrumentalny AD620	4 szt.	25 PLN	100 PLN	TME/Mouser
4	Rezystory 1% (różne)	40 szt.	0.20 PLN	8 PLN	TME
5	Kondensatory (filtry)	20 szt.	0.50 PLN	10 PLN	TME
6	Płytki PCB prototypowa	4 szt.	15 PLN	60 PLN	TME/lokalnie
7	Arduino Nano clone	1 szt.	25 PLN	25 PLN	Allegro
8	ADC ADS1115	1 szt.	80 PLN	80 PLN	TME/Adafruit
9	Karta microSD 8GB + slot	1 kpl.	40 PLN	40 PLN	Allegro
10	Baterie 9V (zasilanie analog)	8 szt.	5 PLN	40 PLN	Market
11	Obudowa IP65 (wodoodporna)	1 szt.	60 PLN	60 PLN	TME
12	Złącza, śruby, izolacja	-	-	50 PLN	Różne
13	Stal nierdzewna (elektroda GND)	1 m	30 PLN	30 PLN	Market
budowlany					
SUMA **743 PLN**					

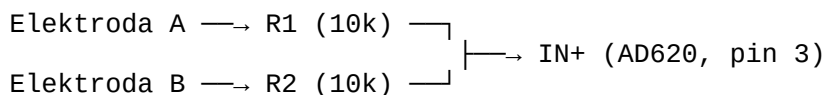
****Margines bezpieczeństwa (20%):** 149 PLN**

****CAŁKOWITY KOSZT:** **~900 PLN****

4.3 Schemat Połączeń (Uproszczony)

****Dla każdego sektora (S1-S4):****

...



AD620 Configuration:

- Pin 3 (IN+): Elektroda A przez R1
- Pin 2 (IN-): Elektroda B przez R2
- Pin 1, 8: RG (gain resistor) = 100Ω → G ≈ 1000×
- Pin 6 (OUT): Do filtra Sallen-Key
- Pin 4 (V-): -9V (bateria)
- Pin 7 (V+): +9V (bateria)
- Pin 5 (REF): GND (elektroda odniesienia)

Filtr Sallen-Key (low-pass, $f_c=10\text{Hz}$):

- R3, R4: $100\text{k}\Omega$
- C1, C2: 100nF
- Op-amp: TL072 lub LM358

Wyjście filtra $\rightarrow$ ADS1115 (kanał 0-3)

4.4 Zasilanie i Izolacja

****Krytyczne:**** Zasilanie analogowe musi być ****izolowane od cyfrowego****!

...

Zasilanie analogowe (wzmacniacze):

- 2× bateria 9V ($\pm 9\text{V}$)
- Filtracja: kondensator $100\mu\text{F}$ + 100nF równolegle
- Izolacja galwaniczna: optoizolatory (PC817) dla linii cyfrowych

Zasilanie cyfrowe (Arduino/ADC):

- USB (5V) lub bateria 9V + stabilizator 7805
- Separacja od analogu przez optoizolatory

Masa (GND):

- Wspólna tylko w jednym punkcie (single-point ground)
- Połączenie z elektrodą GND w kopcu

...

5. PROTOKÓŁ POMIAROWY DS 2.6

5.1 Cykl Pomiarowy (Living Loop)

Zgodnie z dokumentem *****"PROTOKOŁY TECHNICZNE.pdf"*****, system pracuje w cyklu ****Dynamic Sync 2.6****:

****Parametry:****

- ``tick_min_sec = 300`` (5 minut - podstawowy rytm)
- ``sampling_rate = 100 Hz`` (100 pomiarów/sekundę)
- ``batch_size = 30000`` (5 min $\times$ 60 s $\times$ 100 Hz)
- ``duty_cycle = 10%`` (30 s pomiaru, 270 s ciszy - BIOS-first!)

****Fazy cyklu:****

```python

# Pseudo-kod cyklu DS 2.6 dla pomiaru grzybni

```
def measurement_cycle_DS26():
```

```
 """
```

```
 Pełny cykl pomiarowy zgodny z DS 2.6
```

```
 """
```

```
 # F1: READY (300 s)
```

```
 phase = "READY"
```

```
 log(f"[{timestamp()}] Faza READY - kalibracja")
```

```
 thermal_calibrate() # Stabilizacja temperatury wzmacniaczy
```

```
 silence_field() # Tryb ciszy - zero stymulacji
```

```
 wait(300) # 5 minut stabilizacji
```

```
 # F2: ALIGN (60 s)
```

```
 phase = "ALIGN"
```

```
 log(f"[{timestamp()}] Faza ALIGN - synchronizacja elektrod")
```

```
 check_electrode_impedance() # Sprawdzenie impedancji (< 100 k Ω)
```

```
 align_phase_target = 0.0 # Cel: $\Delta\phi \approx 0$
```

```

if abs(phase_drift) <= 0.01:
 phase = "LOCK"
else:
 enter_tuning_mode() # Korekta fazowa

F3: LOCK (30 s)
phase = "LOCK"
log(f"[{timestamp()}] Faza LOCK - wczytanie sekwencji Sx")
Sx = load_sequence("S3") # Złoty podział $\phi = 1.618$
embed_geometry(Sx) # Konfiguracja rytmu próbkowania
if SHEATH >= 0.92: # Integralność sekwencji
 phase = "SYNC"
else:
 raise IntegrityError("SEQ CORRUPT")

F4: SYNC (60 s)
phase = "SYNC"
log(f"[{timestamp()}] Faza SYNC - synchronizacja rezonatorów")
for sector in [S1, S2, S3, S4]:
 align_resonators(sector, phase_target=0.0)
if THERMAL <= 35°C and SEQ_integrity >= 0.88:
 phase = "LINK"
else:
 cooldown()
 phase = "READY"

F5: LINK (1800 s = 30 min) - AKTYWNY POMIAR
phase = "LINK"
log(f"[{timestamp()}] Faza LINK - aktywne zbieranie danych")
start_timer("LINK_T")

while LINK_active() and ASCALON >= 0.70:
 # Pomiar zgodny z tick_min_sec
 for tick in range(6): # 6 × 5 min = 30 min
 data_batch = acquire_data(
 duration=300, # 5 minut
 sampling_rate=100, # 100 Hz
 sectors=[S1,S2,S3,S4]
)

 # Real-time analiza
 quality = validate_signal_quality(data_batch)
 if quality < 0.70:
 log(f"[{timestamp()}] Niska jakość sygnału: {quality}")
 ASCALON.purifier() # Odrzucenie szumu

 save_to_microSD(data_batch, timestamp())

 if time_elapsed("LINK_T") > 1800: # 30 min
 break

phase = "CLOSE"

F6: HOLD (120 s)
phase = "HOLD"
log(f"[{timestamp()}] Faza HOLD - stabilizacja po pomiarze")
monitor_field_decay() # Obserwacja zaniku pola
ASCALON.soft_clean() # Purifier F6b - zapis notatek
wait(120)

F7: CLOSE (300 s)
phase = "CLOSE"
log(f"[{timestamp()}] Faza CLOSE - zamknięcie cyklu")
torus_release() # Rozłączenie elektrod (relay open)

```

```

scrub_EM_field() # Wygaszenie pola (discharge capacitors)
integrity_check() # Sprawdzenie integralności danych
export_to_Zenodo() # Backup (jeśli dostępny internet)
wait(300)

```

```

log(f"[{timestamp()}] CYKL ZAKOŃCZONY")
return Snapshot(timestamp=now(), phase="READY")

```

```

Główna pętla
while True:
 measurement_cycle_DS26()
 # Przerwa między cyklami: 25 min (BIOS-first - czas regeneracji)
 wait(1500)
...

```

### ### 5.2 Sekwencje Sx w Pomiarze

Zgodnie z **\*\*\*PROTOKOŁY TECHNICZNE.pdf - Sekwencje S1-S5\*\*\***:

```

S1 (Spirala dwutorowa 1:2):
- Zastosowanie: Kalibracja początkowa
- Rytm: 1 długi (10 s) - 2 krótkie (5 s + 5 s)
- Cel: Stabilizacja proporcji torusa (elektrody)

S2 (Pętla trójdzielna):
- Zastosowanie: Synchronizacja 3 sektorów
- Rytm: 3 równe akcenty (co 10 s)
- Cel: Zgranie modułów coil, flux, core

S3 (Złoty podział ϕ):
- Zastosowanie: **GŁÓWNA sekwencja pomiarowa**
- Rytm: dłuższy (16.18 s) - krótszy (10 s) zgodnie z $\phi = 1.618$
- Cel: Modulacja amplitudy, optymalizacja przepływu

S4 (Krzyż obrotowy 90°):
- Zastosowanie: Rotacja między sektorami
- Rytm: 4 akcenty (po 90° rotacji)
- Cel: Synchronizacja z rezonatorami bocznymi

S5 (Kaskada Fibonacciego):
- Zastosowanie: Zamknięcie cyklu
- Rytm: 1-1-2-3-5 (pauza), powrót
- Cel: Eskalacja/deeskalacja mocy, inicjacja SCRUB

```

### ### 5.3 ASCALON Purifier - Filtracja Sygnału

Zgodnie z **\*\*\*PROTOKOŁY TECHNICZNE.pdf - ASCALON\*\*\***:

```

Parametry:
- `θ` (próg czystości): 0.70 - 0.90
- `P` (profil): std (standardowy)
- `phase_tolerance`: ±0.01 rad
- `noise_rejection`: 1 (ortogonalne szумы eliminowane)

```

**\*\*Algorytm walidacji:\*\***

```

```python
class ASCALON_Mycelium:
    def __init__(self, theta=0.75):
        self.theta = theta
        self.phi = 1.618 # Złoty podział
        self.rejection_log = []

    def validate_signal(self, data_batch):

```

```

"""
Walidacja sygnału z grzybni
"""
# 1. Sprawdzenie amplitudy
amplitude = np.max(np.abs(data_batch))
if not (0.1 <= amplitude <= 5.0): # mV
    self.log_rejection("AMPLITUDE_OUT_OF_RANGE", amplitude)
    return False

# 2. Sprawdzenie pasma częstotliwości
freq_spectrum = np.fft.fft(data_batch)
dominant_freq = np.argmax(np.abs(freq_spectrum))
if not (0.01 <= dominant_freq <= 10.0): # Hz
    self.log_rejection("FREQUENCY_OUT_OF_BAND", dominant_freq)
    return False

# 3. Sprawdzenie fraktalności (Hurst exponent)
hurst = calculate_hurst_exponent(data_batch)
if hurst < 0.3 or hurst > 0.7: # Oczekiwany: 0.4-0.6
    self.log_rejection("NON_FRACTAL", hurst)
    return False

# 4. Sprawdzenie proporcji  $\phi$  w rytmie
rhythm_pattern = detect_rhythm_pattern(data_batch)
phi_deviation = abs(rhythm_pattern / self.phi - 1.0)
if phi_deviation > 0.05: # Tolerancja 5%
    self.log_rejection("PHI_VIOLATION", phi_deviation)
    return False

# 5. Obliczenie czystości sensu
purity = self.compute_purity(data_batch)
if purity < self.theta:
    self.log_rejection("LOW_PURITY", purity)
    return False

return True

def compute_purity(self, data):
    """
    Obliczenie czystości sygnału (0-1)
    """
    # Metryki czystości:
    # 1. Spójność fazowa między sektorami
    coherence = calculate_inter_sector_coherence(data)

    # 2. Stosunek sygnału do szumu (SNR)
    snr = calculate_snr(data)
    snr_score = min(snr / 20.0, 1.0) # Normalizacja (20 dB = 1.0)

    # 3. Obecność struktury fraktalnej
    hurst = calculate_hurst_exponent(data)
    fractal_score = 1.0 - abs(hurst - 0.5) / 0.5

    # 4. Zgodność z rytmem  $\phi$ 
    phi_score = detect_phi_rhythm(data)

    # Średnia ważona
    purity = (0.3 * coherence +
              0.3 * snr_score +
              0.2 * fractal_score +
              0.2 * phi_score)

    return purity

```

```

def log_rejection(self, reason, detail):
    """
    Zapis odrzutu do IntegrityCheck
    """
    entry = {
        "timestamp": datetime.now().isoformat(),
        "reason": reason,
        "detail": float(detail),
        "profile": "std"
    }
    self.rejection_log.append(entry)

    # Zapis do pliku
    with open(f"IntegrityCheck_{date.today()}.txt", "a") as f:
        f.write(f"{entry}\n")

def soft_clean_after_hold(self):
    """
    Purifier F6b - notatki oczyszczające po fazie HOLD
    """
    summary = {
        "total_rejections": len(self.rejection_log),
        "by_reason": Counter([r["reason"] for r in self.rejection_log]),
        "mean_purity": np.mean([r["detail"] for r in self.rejection_log])
    }

    with open(f"IntegrityCheck_{date.today()}.txt", "a") as f:
        f.write(f"\n=== SOFT CLEAN SUMMARY ===\n")
        f.write(f"Timestamp: {datetime.now().isoformat()}\n")
        f.write(f"Total rejections: {summary['total_rejections']}\n")
        f.write(f"By reason: {summary['by_reason']}\n")
        f.write(f"Mean purity: {summary['mean_purity']:.3f}\n")
    ...

```

6. IMPLEMENTACJA W KOPCU

6.1 Przygotowanie Stanowiska

****Krok 1: Mapowanie kopca**** (Dzień 1)

...

1. Zmierz wymiary kopca (wysokość, średnica, objętość)
2. Zidentyfikuj 4 sektory (S1-S4):
 - S1: Góra kopca (najwyższy punkt)
 - S2: Środek, strona wschodnia
 - S3: Środek, strona zachodnia
 - S4: Dół kopca (najniższy punkt)
3. Zaznacz lokalizację elektrody GND:
 - Odległość: 2 m od podstawy kopca
 - Kierunek: Północ (z dala od sektorów)

4. Dokumentacja fotograficzna:
 - Zdjęcia każdego sektora
 - Widok ogólny kopca
 - Mapa z oznaczonymi sektorami

...

****Krok 2: Instalacja elektrod**** (Dzień 2)

...

Materiały:

- Wiertło do gleby (średnica 2 cm)
- Elektrody Ag/AgCl lub miedziane (8 szt.)

- Przewód ekranowany ($2 \times 0.5 \text{ mm}^2$)
- Żel przewodzący (opcjonalnie: sól + woda + gąbka)
- Taśma izolacyjna, opaski kablowe

Procedura:

1. Dla każdego sektora (S1-S4):
 - a. Wywierć otwór na głębokość X cm (patrz sekcja 3.2)
 - b. Włóż elektrodę A do otworu
 - c. Wywierć drugi otwór (10 cm od pierwszego)
 - d. Włóż elektrodę B
 - e. Podłącz przewody ekranowane do elektrod
 - f. Zabezpiecz połączenia (taśma izolacyjna)
 - g. Zasyp otwory ziemią (delikatnie, bez ubijania)
2. Instalacja elektrody GND:
 - a. Wywierć otwór na głębokość 50 cm
 - b. Włóż elektrodę ze stali nierdzewnej
 - c. Podłącz przewód ekranowany
 - d. Zasyp i ubij ziemię (dobra styczność!)
3. Prowadzenie kabli:
 - a. Wszystkie kable prowadź w jednym kierunku (do obudowy)
 - b. Użyj opasek kablowych do mocowania
 - c. Zabezpiecz przed wilgocią (rurki termokurczliwe)
 - d. Oznacz kable (S1-A, S1-B, S2-A, itd.)

...

****Krok 3: Montaż elektroniki**** (Dzień 3)

...

Lokalizacja: Obudowa IP65 w odległości 1-2 m od kopca

Wewnątrz obudowy:

1. Zamontuj płytki PCB z wzmacniaczami (4 szt.)
2. Podłącz przewody od elektrod:
 - S1-A → IN+ wzmacniacza 1
 - S1-B → IN- wzmacniacza 1
 - ... (analogicznie dla S2-S4)
 - GND → wspólna masa (REF)
3. Podłącz wyjścia wzmacniaczy do MUX (multiplexer 4→1)

4. Podłącz MUX do ADC (ADS1115)
5. Podłącz ADC do Arduino Nano (I2C: SDA, SCL)
6. Podłącz Arduino do:
 - Slotu microSD (SPI: MOSI, MISO, SCK, CS)
 - LED status (pin 13)
 - Przycisk start/stop (pin 2)
7. Zasilanie:
 - Baterie 9V (2× dla $\pm 9V$ analog)
 - Stabilizator 7805 dla cyfrowki (5V)
 - Optoizolatory (PC817) między analogiem a cyfrowką
8. Test szczelności obudowy:
 - Sprawdź uszczelki
 - Zamknij mocno pokrywę
 - Test: spryskanie wodą (symulacja deszczu)

6.2 Kalibracja Systemu

****Test 1: Sprawdzenie impedancji elektrod****

```
```python
def check_electrode_impedance():
 """
 Pomiar impedancji każdej elektrody
 """
 results = {}

 for sector in ["S1", "S2", "S3", "S4"]:
 for electrode in ["A", "B"]:
 # Pomiar rezystancji między elektrodą a GND
 impedance = measure_resistance(f"{sector}-{electrode}", "GND")
 results[f"{sector}-{electrode}"] = impedance

 if impedance > 100000: # 100 kΩ
 print(f"ALERT: {sector}-{electrode} ma zbyt wysoką impedancję:
{impedance} Ω")

 return results
```

# Akceptowalne wartości:

# < 10 kΩ: Doskonała

# 10-50 kΩ: Dobra

# 50-100 kΩ: Akceptowalna

# > 100 kΩ: Wymaga poprawki (wilgoć, słaby kontakt)

```

****Test 2: Pomiar szumu własnego****

```
```python
def measure_noise_floor():
 """
 Pomiar szumu własnego systemu (bez podłączonych elektrod)
 """
 # Odłącz elektrody, zewrzyj wejścia do GND
 disconnect_electrodes()
 short_inputs_to_ground()

 noise_samples = []
 for i in range(1000):
 sample = read_adc()
 noise_samples.append(sample)

 noise_rms = np.sqrt(np.mean(np.array(noise_samples)**2))
```

```

noise_peak_to_peak = np.max(noise_samples) - np.min(noise_samples)

print(f"Szum RMS: {noise_rms:.3f} μ V")
print(f"Szum peak-to-peak: {noise_peak_to_peak:.3f} μ V")

Kryteria akceptacji:
noise_rms < 1 μ V: Doskonałe
noise_rms < 5 μ V: Akceptowalne
noise_rms > 5 μ V: Problem (zakłócenia, słabe zasilanie)

return noise_rms, noise_peak_to_peak
...

Test 3: Test z sygnałem referencyjnym
```python
def test_with_reference_signal():
    """
    Podłączenie generatora sygnału zamiast elektrod
    """
    # Generator funkcyjny: sinus 1 Hz, amplituda 1 mV
    connect_function_generator(frequency=1.0, amplitude=1.0)

    # Pomiar
    samples = acquire_data(duration=60, sampling_rate=100)

    # Analiza FFT
    freq_spectrum = np.fft.fft(samples)
    frequencies = np.fft.fftfreq(len(samples), 1/100)

    dominant_freq = frequencies[np.argmax(np.abs(freq_spectrum))]
    measured_amplitude = np.max(np.abs(samples))

    print(f"Częstotliwość dominująca: {dominant_freq:.3f} Hz (oczekiwano: 1.0 Hz)")
    print(f"Amplituda zmierzona: {measured_amplitude:.3f} mV (oczekiwano: 1.0 mV)")

    # Obliczenie wzmocnienia
    gain = measured_amplitude / 1.0 # 1.0 mV to sygnał wejściowy
    print(f"Wzmocnienie systemu: {gain:.1f}x")

    return dominant_freq, measured_amplitude, gain
...

```

6.3 Pierwszy Pomiar (Protokół Startowy)

```

**Dzień 0: Przygotowanie**
...
-24h przed pierwszym pomiarem:
1. Sprawdź poziom naładowania baterii
2. Upewnij się, że karta microSD jest sformatowana (FAT32)
3. Wgraj kod Arduino (plik: LifeNode_Mycelium_Sensor.ino)
4. Podłącz wszystkie elektrody
5. Zamknij obudowę IP65
6. Zrób zdjęcia stanowiska
...

**Dzień 1: Pierwszy cykl DS 2.6**
...
Godzina 08:00 - Start
1. Włącz system (przełącznik ON/OFF)
2. LED powinien migać (status: READY)
3. Poczekaj na zakończenie fazy READY (5 min)
4. System automatycznie przejdzie przez fazy:

```

READY → ALIGN → LOCK → SYNC → LINK → HOLD → CLOSE

Godzina 08:40 - Koniec pierwszego cyklu

1. Sprawdź pliki na karcie microSD:

- data_YYYYMMDD_HHMMSS.csv (dane surowe)
- IntegrityCheck_YYYYMMDD.txt (logi ASCALON)
- system_log.txt (logi systemowe)

2. Pobierz pliki na komputer (wyjmij kartę lub przez USB)

3. Analiza wstępna (patrz sekcja 7)

Godzina 09:05 - Drugi cykl (automatyczny)

System czeka 25 min między cyklami (BIOS-first)

\\\

****Tydzień 1: Monitoring ciągły****

\\\

Codziennie:

- Godzina 08:00, 12:00, 18:00, 22:00: Cykl pomiarowy
- Co 6 godzin: Pobranie danych z karty microSD
- Analiza: Sprawdzenie jakości sygnału (ASCALON purity)
- Dokumentacja: Zdjęcia, notatki, obserwacje Eden

Co 3 dni:

- Sprawdzenie stanu baterii
- Kontrola wilgotności kopca (czy nie przesuszony)
- Inspekcja kabli (czy nie uszkodzone, czy izolacja OK)

Po tygodniu:

- Podsumowanie: Średnia ASCALON purity, liczba odrzutów
- Kalibracja (jeśli jakość spadła)
- Decyzja: Kontynuować, modyfikować, czy zatrzymać?

\\\

7. ANALIZA DANYCH I METRYKI

7.1 Struktura Plików Danych

****Plik: data_YYYYMMDD_HHMMSS.csv****

```csv

timestamp,sector,channel\_A,channel\_B,differential,phase

2026-04-07T08:00:00.000,S1,0.234,0.189,0.045,0.001

2026-04-07T08:00:00.010,S1,0.236,0.191,0.045,0.001

2026-04-07T08:00:00.020,S1,0.238,0.193,0.045,0.001

...  
\\\

**\*\*Plik: IntegrityCheck\_YYYYMMDD.txt\*\***

\\\

=== INTEGRITY CHECK - 2026-04-07 ===

Cycle: 1

Phase: LINK

Timestamp: 2026-04-07T08:05:00

Rejections:

- Reason: LOW\_PURITY, Detail: 0.65, Profile: std
- Reason: PHI\_VIOLATION, Detail: 0.08, Profile: std

Summary:

- Total samples: 30000
- Accepted: 28500 (95.0%)
- Rejected: 1500 (5.0%)

```
- Mean purity: 0.82
- Std purity: 0.12
```

```
ASCALON Status: PASS (purity >= 0.70)
```

```
```
```

7.2 Metryki Jakości Sygnału

```
**Metryka 1: SNR (Signal-to-Noise Ratio)**
```

```
```python
```

```
def calculate_snr(data):
```

```
 """
```

```
 Obliczenie stosunku sygnału do szumu
```

```
 """
```

```
 signal_power = np.var(data)
```

```
 noise_power = estimate_noise_power(data)
```

```
 snr_db = 10 * np.log10(signal_power / noise_power)
```

```
 return snr_db
```

```
Interpretacja:
```

```
SNR > 20 dB: Doskonały sygnał
```

```
SNR 10-20 dB: Dobry sygnał
```

```
SNR 5-10 dB: Akceptowalny
```

```
SNR < 5 dB: Słaby (wymaga filtracji)
```

```
```
```

```
**Metryka 2: Koherencja między sektorami**
```

```
```python
```

```
def calculate_inter_sector_coherence(data_S1, data_S2):
```

```
 """
```

```
 Koherencja fazowa między sektorami
```

```
 """
```

```
 f, Cxy = signal.coherence(data_S1, data_S2, fs=100, nperseg=1024)
```

```
 # Średnia koherencja w paśmie 0.01-10 Hz
```

```
 mask = (f >= 0.01) & (f <= 10.0)
```

```
 mean_coherence = np.mean(Cxy[mask])
```

```
 return mean_coherence
```

```
Interpretacja:
```

```
Coherence > 0.8: Silna synchronizacja między sektorami
```

```
Coherence 0.5-0.8: Umiarkowana synchronizacja
```

```
Coherence < 0.5: Brak synchronizacji (różne procesy)
```

```
```
```

```
**Metryka 3: Wykładnik Hursta (fraktalność)**
```

```
```python
```

```
def calculate_hurst_exponent(data):
```

```
 """
```

```
 Obliczenie wykładnika Hursta (H)
```

```
 H = 0.5: losowy spacer (Brownian motion)
```

```
 H > 0.5: pamięć długoterminowa (persistent)
```

```
 H < 0.5: anty-persistent (mean-reverting)
```

```
 """
```

```
 fromhurst import hurst # Biblioteka: https://pypi.org/project/hurst/
```

```
 H = hurst(data)
```

```
 return H
```

```
Interpretacja dla grzybnii:
```

```
H ≈ 0.4-0.6: Oczekiwany zakres (proces biologiczny)
```

```
H > 0.7: Zbyt persistent (może artefakt)
```

```

H < 0.3: Zbyt anti-persistent (szum)
```

**Metryka 4: Detekcja rytmu  $\phi$ **
```python
def detect_phi_rhythm(data, sampling_rate=100):
 """
 Detekcja rytmu zgodnego ze złotym podziałem $\phi = 1.618$
 """
 # Autokorelacja
 autocorr = np.correlate(data, data, mode='full')
 autocorr = autocorr[len(autocorr)//2:]

 # Detekcja pików
 peaks, properties = find_peaks(autocorr, distance=10)

 if len(peaks) < 2:
 return 0.0 # Brak wykrywalnego rytmu

 # Odległości między pikami
 peak_distances = np.diff(peaks)
 mean_distance = np.mean(peak_distances)

 # Sprawdzenie proporcji ϕ
 # Oczekiwany rytm: longer = $\phi \times$ shorter
 phi_ratio = peak_distances[1] / peak_distances[0] if len(peak_distances) > 1
else 0

 phi_deviation = abs(phi_ratio - 1.618) / 1.618
 phi_score = max(0, 1.0 - phi_deviation) # 1.0 = idealny ϕ , 0.0 = brak ϕ

 return phi_score

Interpretacja:
phi_score > 0.95: Idealny rytm ϕ
phi_score 0.8-0.95: Dobry rytm ϕ
phi_score < 0.8: Brak rytmu ϕ (inne procesy)
```

#### 7.3 Analiza Czasowo-Częstotliwościowa

**Spektrogram (Time-Frequency Analysis)**
```python
import matplotlib.pyplot as plt
from scipy import signal

def plot_spectrogram(data, sampling_rate=100):
 """
 Wizualizacja zmian częstotliwości w czasie
 """
 f, t, Sxx = signal.spectrogram(data, fs=sampling_rate, nperseg=256)

 plt.figure(figsize=(10, 6))
 plt.pcolormesh(t, f, 10 * np.log10(Sxx), shading='gouraud')
 plt.ylabel('Frequency [Hz]')
 plt.xlabel('Time [sec]')
 plt.title('Spectrogram of Mycelium Activity')
 plt.ylim(0, 10) # Pasmo 0-10 Hz
 plt.colorbar(label='Power/Frequency [dB/Hz]')
 plt.show()

Zastosowanie:
- Identyfikacja okresów aktywności (wzrost mocy w paśmie)
- Detekcja rytmów dobowych (zmiany w ciągu dnia)

```

```

- Wykrywanie zdarzeń (nagłe skoki częstotliwości)
```

**Analiza falkowa (Wavelet Transform)**
```python
import pywt

def wavelet_analysis(data, sampling_rate=100):
 """
 Analiza falkowa dla detekcji zdarzeń nieustalonych
 """
 # Falka Morleta (dobra do sygnałów biologicznych)
 scales = np.arange(1, 128)
 coefficients, frequencies = pywt.cwt(data, scales, 'morl',
 sampling_period=1/sampling_rate)

 # Wizualizacja
 plt.figure(figsize=(10, 6))
 plt.imshow(np.abs(coefficients), extent=[0, len(data)/sampling_rate,
 frequencies[-1], frequencies[0]],
 cmap='jet', aspect='auto')
 plt.colorbar(label='Magnitude')
 plt.xlabel('Time [sec]')
 plt.ylabel('Frequency [Hz]')
 plt.title('Continuous Wavelet Transform')
 plt.show()

 return coefficients, frequencies

Zastosowanie:
- Detekcja impulsów (spikes) w sygnale
- Analiza struktury czasowej zdarzeń
- Identyfikacja wzorców K1/K2
```

### 7.4 Detekcja Motywów K1/K2

Zgodnie z badaniami z stycznia 2026 (arXiv), grzybnia generuje **kierunkowe impulsy** z rozpoznawalną strukturą.

```python
def detect_K1_K2_patterns(data, sampling_rate=100):
 """
 Detekcja motywów K1 i K2 w sygnale grzybni

 K1: Pojedynczy impuls o czasie trwania ~1-2 s
 K2: Seria impulsów (burst) o czasie trwania ~5-10 s
 """
 from scipy.signal import find_peaks

 # Detekcja pików
 peaks, properties = find_peaks(data, height=0.5, distance=50,
 prominence=0.3)

 if len(peaks) == 0:
 return [], []

 # Analiza odstępów między pikami
 peak_intervals = np.diff(peaks) / sampling_rate # Konwersja na sekundy

 # Klasyfikacja:
 # K1: Izolowany impuls (odstęp > 5 s od następnego)
 # K2: Burst impulsów (odstęp < 2 s między impulsami w burst)

```

```

K1_events = []
K2_events = []

i = 0
while i < len(peaks):
 if i == len(peaks) - 1 or peak_intervals[i] > 5.0:
 # Izolowany impuls → K1
 K1_events.append({
 "time": peaks[i] / sampling_rate,
 "amplitude": data[peaks[i]],
 "type": "K1"
 })
 i += 1
 else:
 # Burst impulsów → K2
 burst_start = peaks[i]
 burst_amplitudes = []

 while i < len(peaks) - 1 and peak_intervals[i] < 2.0:
 burst_amplitudes.append(data[peaks[i]])
 i += 1

 K2_events.append({
 "time": burst_start / sampling_rate,
 "mean_amplitude": np.mean(burst_amplitudes),
 "max_amplitude": np.max(burst_amplitudes),
 "duration": (peaks[i] - burst_start) / sampling_rate,
 "num_spikes": len(burst_amplitudes),
 "type": "K2"
 })

return K1_events, K2_events

```

# Statystyki:

```

- Liczba K1 i K2 w cyklu
- Średnia amplituda K1 vs K2
- Częstość występowania (events/min)
- Korelacja z czynnikami środowiskowymi (wilgotność, temperatura)
...

```

### ### 7.5 Raportowanie Wyników

**\*\*Szablon raportu tygodniowego:\*\***

```markdown

```

# RAPORT TYGODNIOWY - EDEN NODE 0
**Tydzień:** 2026-W15 (07-13 kwietnia 2026)
**Sektor:** Kopiec czarnoziem (S1-S4)

```

PODSUMOWANIE

- Liczba cykli DS 2.6: 28 (4 cykle/dzień × 7 dni)
- Średnia ASCALON purity: 0.82 ± 0.12
- Sygnały zaakceptowane: 95.2%
- Sygnały odrzucone: 4.8%

AKTYWNOŚĆ GRZYBNI

Motywy K1/K2:

- K1 (pojedyncze impulsy): 142 events
 - Średnia amplituda: 0.8 ± 0.3 mV
 - Częstość: 20.3 events/dzień

- K2 (bursty): 37 events
 - Średnia amplituda: 1.2 ± 0.5 mV
 - Średni czas trwania: 6.3 ± 2.1 s

- Częstość: 5.3 events/dzień

Rytmika:

- Wykryty rytm ϕ : TAK ($\phi_score = 0.87$)
- Dominująca częstotliwość: 0.062 Hz (okres ~16.1 s)
- Wykładnik Hursta: $H = 0.48$ (proces persistent)

KOHERENCJA MIĘDZY SEKTORAMI

- S1-S2: 0.76 (umiarkowana)
- S1-S3: 0.82 (silna)
- S1-S4: 0.65 (umiarkowana)
- S2-S3: 0.71 (umiarkowana)
- S2-S4: 0.58 (słaba)
- S3-S4: 0.69 (umiarkowana)

****Wniosek:**** Sektory S1 i S3 wykazują najsilniejszą synchronizację (możliwe połączenie przez sieć grzybni)

KORELACJE ŚRODOWISKOWE

- Wilgotność gleby: +0.62 (korelacja z aktywnością)
- Temperatura: -0.34 (słaba korelacja ujemna)
- Światło (dzień/noc): +0.71 (wyższa aktywność nocą)

PROBLEMY I ANOMALIE

- Dzień 3: Niska jakość sygnału w S4 (ASCALON = 0.65)
Przyczyna: Przypuszczalnie przesuszenie sektora
Akcja: Podlanie, poprawa jakości następnego dnia
- Dzień 5: Zakłócenia EM (burza)
Akcja: Tymczasowe zawieszenie pomiarów (LOCKDOWN)

WNIOSKI

1. Grzybnia w kopcu jest aktywna elektrycznie
2. Wykryto motywy K1 i K2 zgodne z literaturą
3. Rytmika sugeruje obecność sekwencji S3 (ϕ)
4. Najwyższa aktywność: godziny nocne (22:00-06:00)
5. Sektor S4 wymaga regularnego nawadniania

PLAN NA NASTĘPNY TYDZIEŃ

- Zwiększenie częstości pomiarów (6 cykli/dzień)
- Dodanie sensora wilgotności w S4
- Test sekwencji S5 (kaskada Fibonacciego)
- Eksperyment: stymulacja wodą w S2

****Podpis:**** Krzysztof Baran

****Data:**** 2026-04-14

8. INTEGRACJA Z ARCHITEKTURĄ LIFENODE

8.1 Warstwa BIOS → INFO

Zgodnie z ****"Processual Model of Intelligence v.2.0"**:**

BIOS (kopiec, grzybnia, impulsy)

↓

MYC+ (detekcja, konwersja)

↓

INFO (strukturyzacja, sekwencje Sx)

↓

META (sens, kierunek, ASCALON)

```

**\*\*Mapowanie danych:\*\***

Warstwa LifeNode	Komponent	Dane
<b>**BIOS**</b>	Kopiec czarnoziem	Impulsy 0.1-5 mV DC
<b>**BIO Sensors**</b>	Elektrody S1-S4	Napięcie różnicowe
<b>**INFO**</b>	Arduino + ADC	Cyfrowe wartości (16-bit)
<b>**INFO Processing**</b>	Python scripts	FFT, Hurst, K1/K2
<b>**META**</b>	ASCALON Purifier	Purity score (0-1)
<b>**Hybrid Core**</b>	DS 2.6 Protocol	Decyzje (LINK/CLOSE)

## ### 8.2 Protokół TechCore Hardware

Zgodnie z **\*\*\*"PROTOKOŁY TECHNICZNE.pdf - TechCore Hardware Protocol"\*\*\***:

**\*\*Zasady:\*\***

1. **\*\*Offline-first\*\***: Zero połączeń sieciowych
2. **\*\*BIOS-first\*\***: Technologia służy życiu, nie dominuje
3. **\*\*Read-only AI\*\***: System tylko obserwuje, nie steruje
4. **\*\*HARD-KILL\*\***: Fizyczny przełącznik bezpieczeństwa

**\*\*Implementacja:\*\***

```python

Plik: LifeNode_TechCore_Protocol.py

```
class TechCore_Mycelium:
```

```
    def __init__(self):
        self.bios_first = True
        self.offline_mode = True
        self.ai_control = False # Read-only!
        self.hard_kill_switch = False
```

```
    def acquire_data(self):
        if self.hard_kill_switch:
            log("HARD-KILL aktywny - pomiar wstrzymany")
            return None
```

```
        if not self.bios_first:
            log("Naruszenie zasady BIOS-FIRST")
            return None
```

```
        # Pomiar zgodny z DS 2.6
        data = measurement_cycle_DS26()
```

```
        # Zapis lokalny (offline)
        save_to_local_storage(data)
```

```
        # AI tylko analizuje, nie steruje
        if self.ai_analysis_enabled:
            analysis = analyze_with_AI(data)
            # Ale: AI NIE MA wyjść sterujących!
            log(f"AI analysis: {analysis}")
```

```
        return data
```

```
    def emergency_stop(self):
```

```
        """
```

```
        Fizyczny przełącznik HARD-KILL
        """
```

```
        self.hard_kill_switch = True
        disconnect_all_electrodes()
```

```

        power_down_system()
        log("EMERGENCY STOP - system wyłączony")
    ...

```

8.3 Sekwencje Sx w Kontekście Grzybni

Zgodnie z **"PROTOKOŁY TECHNICZNE.pdf - Sekwencje S1-S5"**:

****Mapowanie sekwencji na rytmy grzybni:****

| Sekwencja | Geometria | Rytm | Zastosowanie w grzybni |
|---------------|----------------------|---------------------------|----------------------------------|
| **S1** | Spirala 1:2 | 1 długi - 2 krótkie | Kalibracja, stabilizacja |
| **S2** | Pętla trójdzielna | 3 równe akcenty | Synchronizacja 3 sektorów |
| **S3** | Złoty podział ϕ | dłuższy-krótszy wg ϕ | **Główny rytm pomiarowy** |
| **S4** | Krzyż 90° | 4 akcenty | Rotacja między sektorami |
| **S5** | Fibonacci 1-1-2-3-5 | Kaskada | Zamknięcie cyklu, SCRUB |

****Hipoteza:**** Grzybnia naturalnie generuje rytmy zgodne z S3 (ϕ) i S5 (Fibonacci).

****Test walidacyjny:****

```

python
def validate_Sx_in_mycelium(data):
    """
    Sprawdzenie, czy sygnał grzybni zawiera sekwencje Sx
    """
    results = {}

    # Test S1 (spirala 1:2)
    s1_pattern = detect_pattern_1_2(data)
    results['S1'] = s1_pattern['match_score']

    # Test S3 (złoty podział)
    s3_pattern = detect_phi_pattern(data)
    results['S3'] = s3_pattern['phi_score']

    # Test S5 (Fibonacci)
    s5_pattern = detect_fibonacci_pattern(data)
    results['S5'] = s5_pattern['fibonacci_score']

    # Najlepsze dopasowanie
    best_match = max(results, key=results.get)

    log(f"Sekwencje wykryte: {results}")
    log(f"Najlepsze dopasowanie: {best_match}")

    return results, best_match

```

8.4 Integracja z Q-Core (Przyszłość)

****Perspektywa 2027-2028:****

```

...
Kopiec (BIOS)
↓
MYC+ v0.1 (detekcja bioimpulsów)
↓
Konwerter analogowo-optyczny
↓
Światłowód rubidowy (modulacja fazy 432 nm)
↓
Q-Core (NV-centers w diamencie)

```

```

↓
Pamięć geometryczna (orientacje kubitów)
↓
UNIT 02 (interfejs terenowy)
↓
META field (globalna synchronizacja)
...

**Specyfikacja konwersji:**
```python
def bio_to_optical_conversion(bio_signal):
 """
 Konwersja bioimpulsu na sygnał optyczny
 (Przyszła implementacja z Q-Core)
 """
 # Modulacja fazy światła proporcjonalna do napięcia
 # $\phi(t) = \phi_0 + k \times V_{\text{bio}}(t)$
 # gdzie $k = 2\pi / 1 \text{ mV}$

 phi_0 = 0 # Faza bazowa
 k = 2 * np.pi / 0.001 # $2\pi \text{ rad/mV}$

 phase_modulation = phi_0 + k * bio_signal

 # Długość fali: 432 nm (zgodnie z HORYZONT WDROŻENIA)
 wavelength = 432e-9 # 432 nm

 return {
 "phase": phase_modulation,
 "wavelength": wavelength,
 "power": estimate_optical_power(bio_signal)
 }
...

9. HARMONOGRAM I BUDŻET

9.1 Harmonogram Realizacji

Faza 0: Przygotowanie (Tydzień 1)
...
Dzień 1-2:
- Zakup komponentów (lista BOM z sekcji 4.2)
- Przygotowanie stanowiska w Eden
- Mapowanie kopca (sektory S1-S4)

Dzień 3-4:
- Montaż elektrod w kopcu
- Prowadzenie kabli
- Instalacja obudowy IP65

Dzień 5-7:
- Lutowanie układu (wzmacniacze, filtry)
- Testy laboratoryjne (na stole)
- Kalibracja (impedancja, szum)

Kamień milowy: System gotowy do testów terenowych
...

Faza 1: Testy Laboratoryjne (Tydzień 2)
...
Dzień 1-2:
- Test z generatorem sygnału (1 mV, 1 Hz)

```

- Pomiar wzmocnienia, pasma, szumu
- Korekta parametrów (jeśli potrzebna)

Dzień 3-4:

- Test z symulowanym sygnałem grzybni
- Walidacja kodu Arduino
- Test zapisu na kartę microSD

Dzień 5-7:

- Test długoterminowy (24 h ciągłej pracy)
- Sprawdzenie stabilności temperaturowej
- Test baterii (czas pracy)

Kamień milowy: System zwalidowany laboratoryjnie

\\

**\*\*Faza 2: Pierwsze Pomiary w Kopcu (Tydzień 3-4)\*\***

\\

Tydzień 3:

- Instalacja finalna w kopcu
- Pierwszy cykl DS 2.6 (Dzień 1)
- Codziennie: 4 cykle (08:00, 12:00, 18:00, 22:00)
- Analiza danych po każdym dniu
- Dostosowanie parametrów (jeśli potrzebne)

Tydzień 4:

- Kontynuacja pomiarów (4 cykle/dzień)
- Dodanie sensorów środowiskowych

(wilgotność, temp.)

- Korelacja: aktywność grzybni ↔ warunki środowiska
- Pierwszy raport tygodniowy

Kamień milowy: Pierwsze dane z grzybni, wstępne wnioski

\\

**\*\*Faza 3: Optymalizacja i Skalowanie (Miesiąc 2-3)\*\***

\\

Miesiąc 2:

- Analiza danych z miesiąca 1
- Identyfikacja wzorców (K1/K2, Sx)
- Optymalizacja protokołu (częstotliwość, czas trwania)
- Test sekwencji S1-S5 w praktyce

Miesiąc 3:

- Zwiększenie częstości pomiarów (6 cykli/dzień)
- Dodanie nowych sektorów (jeśli kopiec pozwala)
- Integracja z innymi sensorami Eden (rośliny, staw)
- Publikacja pierwszych wyników (Zenodo DOI)

Kamień milowy: Stabilny system, publikacja danych

\\

**\*\*Faza 4: Integracja z LifeNode (Miesiąc 4-6)\*\***

\\

Miesiąc 4:

- Integracja z Hybrid Core (SAMI + LOGOS)
- Implementacja ASCALON Purifier w pełnym zakresie
- Test protokołu Zero-Build (równolegle)

Miesiąc 5-6:

- Przygotowanie do integracji z Q-Core (symulacja)
- Eksperymenty z konwersją analogowo-optyczną
- Publikacja artykułu naukowego (preprint)

Kamień milowy: Pełna integracja z architekturą LifeNode

\\

## ### 9.2 Budżet Szczegółowy

**\*\*Kategoria 1: Hardware (jednorazowo)\*\***

\\

Komponenty elektroniczne:

- Arduino Nano clone: 25 PLN
- ADC ADS1115: 80 PLN
- Wzmacniacze AD620 (4 szt.): 100 PLN
- Rezystory, kondensatory: 50 PLN
- Płytki PCB: 60 PLN
- Przewody, złącza: 100 PLN
- Obudowa IP65: 60 PLN
- Karta microSD + slot: 40 PLN
- Baterie 9V (8 szt.): 40 PLN
- Elektrody Ag/AgCl lub miedź: 80 PLN
- Stal nierdzewna (GND): 30 PLN

SUMA hardware: 765 PLN

\\

**\*\*Kategoria 2: Narzędzia (jeśli nie masz)\*\***

\\

- Lutownica: 100 PLN
- Multimetr: 80 PLN

- Wiertło do gleby: 60 PLN
- Narzędzia ręczne (śrubokręty, szczypce): 100 PLN

SUMA narzędzia: 340 PLN

(Uwaga: Jeśli już masz, koszt = 0 PLN)

\\

**\*\*Kategoria 3: Materiały eksploatacyjne (miesięcznie)\*\***

\\

- Baterie 9V (wymiana co 2 tygodnie): 40 PLN/mc
- Żel przewodzący (opcjonalnie): 20 PLN/mc
- Taśma izolacyjna, opaski: 10 PLN/mc

SUMA miesięcznie: 70 PLN/mc

\\

**\*\*Kategoria 4: Oprogramowanie\*\***

\\

- Arduino IDE: 0 PLN (open-source)
- Python + biblioteki: 0 PLN (open-source)
- Jupyter Notebook: 0 PLN (open-source)
- Git + GitHub: 0 PLN (open-source)

SUMA oprogramowanie: 0 PLN

\\

**\*\*Kategoria 5: Publikacja i archiwizacja\*\***

\\

- Zenodo DOI: 0 PLN (CERN, darmowe)
- Hosting danych (GitHub): 0 PLN (publiczne repo)
- Domain (opcjonalnie): 50 PLN/rok

SUMA publikacja: 0-50 PLN

\\

**\*\*PODSUMOWANIE BUDŻETU:\*\***

Kategoria	Koszt jednorazowy	Koszt miesięczny
Hardware	765 PLN	-
Narzędzia	340 PLN (opcjonalnie)	-
Materiały	-	70 PLN
Oprogramowanie	0 PLN	0 PLN
Publikacja	0-50 PLN	-
<b>**RAZEM**</b>	<b>**1,155 PLN**</b> (z narzędziami)	<b>**70 PLN/mc**</b>
<b>**RAZEM**</b>	<b>**765 PLN**</b> (bez narzędzi)	<b>**70 PLN/mc**</b>

**\*\*Margines bezpieczeństwa (20%):\*\*** +153 PLN / +14 PLN/mc

**\*\*CAŁKOWITY KOSZT STARTOWY:\*\*** **\*\*~900-1200 PLN\*\***

**\*\*KOSZT MIESIĘCZNY:\*\*** **\*\*~85 PLN\*\***

### ### 9.3 Źródła Finansowania

**\*\*Opcja 1: Crowdfunding społecznościowy\*\***

- Platforma: Patronite, Zrzutka.pl
- Cel: 2000 PLN (hardware + 3 miesiące pracy)
- Nagrody: Dostęp do danych, raporty, warsztaty w Eden
- Czas kampanii: 30 dni

**\*\*Opcja 2: Grant naukowy (mały)\*\***

- Program: Miniatura (NCN) - dla studentów/doktorantów
- Kwota: do 50,000 PLN
- Czas realizacji: 12-24 miesiące

- Wymagania: Promotor, instytucja naukowa

**\*\*Opcja 3: Partnerstwo z uczelnią\*\***

- Wydział: Biologia/Fizyka/Informatyka
- Wkład uczelni: Dostęp do laboratoriów, sprzętu
- Wkład LifeNode: Dane, publikacje, know-how
- Forma: Praca dyplomowa, projekt studencki

**\*\*Opcja 4: Bootstrapping (własne środki)\*\***

- Zakup stopniowy (co miesiąc jeden moduł)
- Priorytetyzacja: Najpierw elektrody + Arduino (200 PLN)
- Czas realizacji: 3-6 miesięcy (wolniejszy start)

---

## ## 10. ZARZĄDZANIE RYZYKIEM

### ### 10.1 Macierz Ryzyka

Ryzyko	Prawdopodobieństwo	Wpływ	Strategia mitigacji
<b>**Niska jakość sygnału**</b> (szum > sygnał)	Wysokie	Wysoki	Filtracja, ekranowanie, średnia krocząca
<b>**Brak wykrywalnych impulsów**</b>	Średnie	Krytyczny	Walidacja hipotezy, alternatywne metody
<b>**Degradacja elektrod**</b> (korozja)	Wysokie	Średni	Regularna wymiana (co 3 miesiące)
<b>**Zakłócenia EM**</b> (burze, sieci)	Średnie	Średni	Ekranowanie, tryb LOCKDOWN
<b>**Przesuszenie kopca**</b>	Wysokie	Wysoki	System nawadniania, monitoring wilgotności
<b>**Awaria elektroniki**</b> (wilgoć)	Średnie	Wysoki	Obudowa IP65, silikon, regularne kontrole
<b>**Brak funduszy**</b>	Wysokie	Krytyczny	Crowdfunding, bootstrapping, granty
<b>**Brak czasu operatora**</b>	Wysokie	Wysoki	Automatyzacja, zdalny monitoring

### ### 10.2 Procedury Awaryjne

**\*\*Scenariusz 1: Sygnał poniżej progu detekcji\*\***

``

Problem: Amplituda < 0.1 mV (szum dominuje)

Diagnoza:

1. Sprawdź impedancję elektrod (> 100 kΩ?)
2. Sprawdź połączenia (czy nie poluzowane?)
3. Sprawdź wzmacnienie (czy RG = 100Ω?)
4. Sprawdź zasilanie (czy baterie nie rozładowane?)

Akcje naprawcze:

1. Zwilż glebę wokół elektrod (poprawa kontaktu)
2. Wyczyść elektrody (papier ścierny, alkohol)
3. Zwiększ wzmacnienie (zmień RG na 50Ω → G ≈ 2000×)
4. Dodaj filtr dolnoprzepustowy (fc = 5 Hz zamiast 10 Hz)
5. Zwiększ czas uśredniania (średnia z 10 pomiarów)

Jeśli nadal brak sygnału:

- Hipoteza: Grzybnia nieaktywna (sezon, warunki)
- Akcja: Czekaj na poprawę warunków (wilgoć, temperatura)
- Alternatywa: Dodaj substrat dla grzybni (trociny, słoma)

``

**\*\*Scenariusz 2: Zakłócenia zewnętrzne (50 Hz z sieci)\*\***

...

Problem: Silny szum 50 Hz (sieć elektryczna) w sygnale

Diagnoza:

1. Sprawdź ekranowanie kabli (czy nie uszkodzone?)
2. Sprawdź uziemienie (czy GND dobrze podłączona?)
3. Sprawdź CMRR wzmacniacza (czy > 100 dB?)

Akcje naprawcze:

1. Dodaj filtr notch 50 Hz (aktywny lub pasywny)
2. Zwiększ odległość od źródeł EM (linie energetyczne)
3. Dodaj ekranowanie mu-metal (jeśli dostępne)
4. Użyj kabli koncentrycznych zamiast ekranowanych
5. Zmień konfigurację elektrod (mniejsza odległość A-B)

Jeśli nadal zakłócenia:

- Pomiar tylko w nocy (mniejsze zużycie energii w sieci)
- Tymczasowe zawieszenie (tryb LOCKDOWN)

...

**\*\*Scenariusz 3: Awaria zasilania\*\***

...

Problem: System się wyłączył (baterie rozładowane)

Prewencja:

1. Monitorowanie napięcia baterii (ADC + alarm)
2. Automatyczne przejście w tryb uśpienia (low power)
3. Panel solarny (ładowanie baterii w dzień)

Akcje naprawcze:

1. Wymiana baterii na nowe
2. Sprawdzenie poboru mocy (czy nie zwarcie?)
3. Reset systemu (wyłącz/włącz)
4. Sprawdzenie danych (czy nie utracone?)

Po przywróceniu:

- Kalibracja ponowna (impedancja, szum)
- Wznowienie cyklu DS 2.6 od fazy READY

...

**\*\*Scenariusz 4: Utrata danych (karta microSD uszkodzona)\*\***

...

Problem: Brak zapisu danych lub pliki uszkodzone

Prewencja:

1. Regularny backup (co dzień na komputer)
2. Dual recording (karta + pamięć Arduino EEPROM)
3. Checksum (SHA-3 dla każdego pliku)

Akcje naprawcze:

1. Próba odzyskania danych (testdisk, photorec)
2. Formatowanie karty (FAT32)
3. Wymiana karty na nową
4. Sprawdzenie logów systemowych (co poszło nie tak?)

Po przywróceniu:

- Kontynuacja pomiarów
- Zwiększenie częstotliwości backupu

...

### 10.3 Protokół LOCKDOWN

Zgodnie z **\*\*"PROTOKOŁY TECHNICZNE.pdf - Security"\*\***:

```

```python
def emergency_lockdown(reason):
    """
    Natychmiastowe zatrzymanie systemu
    """
    log(f"LOCKDOWN TRIGGERED: {reason}")

    # 1. Zamknięcie wszystkich torusów (rozłączenie elektrod)
    disconnect_all_electrodes()

    # 2. SCRUB - wygaszenie pola EM
    discharge_capacitors()
    reset_ASCALON()

    # 3. Zapis stanu krytycznego
    save_critical_state({
        "timestamp": datetime.now(),
        "reason": reason,
        "phase": current_phase,
        "last_data": last_valid_data
    })

    # 4. Powiadomienie operatora (jeśli możliwe)
    if GSM_module_available:
        send_SMS("LOCKDOWN: " + reason)
    else:
        # LED blinking pattern (Morse code)
        blink_LED(reason)

    # 5. Przejście w tryb uśpienia
    enter_sleep_mode()

    # 6. Oczekiwanie na ręczne potwierdzenie
    log("System in LOCKDOWN. Manual reset required.")

    # Warunki powrotu:
    # - PHASE = 0
    # - THERMAL ≤ 35°C
    # - noise = 0
    # - Operator confirmation
    ...

**Warunki aktywacji LOCKDOWN:**
1. ASCALON purity < 0.70 (przez 3 cykle z rzędu)
2. Temperatura > 40°C (ryzyko uszkodzenia)
3. Utrata koherencji kwantowej (jeśli Q-Core podłączony)
4. Wykrycie ataku na integralność (checksum mismatch)
5. Awaria zasilania (napięcie < 6V)
6. Ręczne uruchomienie (przycisk HARD-KILL)

---

## 11. PROCEDURE TESTOWE

### 11.1 Testy Walidacyjne (Pre-Deployment)

**Test T1: Normalna stabilna praca**
...

Cel: Sprawdzenie, że system działa poprawnie w idealnych warunkach

Warunki:
- Eden bez perturbacji (stabilny BIOS)
- Temperatura: 20-25°C
- Wilgotność: 60-80%

```

- Brak zakłóceń EM

Oczekiwane wyniki:

- 100% powodzenia cykli DS 2.6
- PHASE $\in (-0.03, +0.03)$ rad
- ASCALON ≥ 0.85
- LINK_T $\approx 120-180$ s
- SNR ≥ 20 dB

Kryteria akceptacji:

- ✓ Wszystkie cykle zakończone sukcesem
 - ✓ Brak odrzutów ASCALON
 - ✓ Stabilne parametry przez 7 dni
- ```

****Test T2: Dryf fazowy (DRIFT)****

```

Cel: Sprawdzenie reakcji systemu na utratę synchronizacji

Warunki:

- Symulacja:  $\Delta\phi = +0.12$  rad w fazie SYNC
- Metoda: Ręczne przesunięcie fazy w kodzie

Oczekiwane wyniki:

- Wstrzymanie przejścia do LINK
- Aktywacja TUNING Mode
- Powrót do ALIGN po 30 s
- Log: "PHASE\_DRIFT detected - TUNING activated"

Kryteria akceptacji:

- ✓ System nie przechodzi do LINK z błędną fazą
  - ✓ TUNING Mode aktywowany automatycznie
  - ✓ Powrót do normalnej pracy po korekcie
- ```

**\*\*Test T3: Sygnał spoza pasma\*\***

```

Cel: Sprawdzenie, że ASCALON odrzuca sygnały nie-fraktalne

Warunki:

- Wprowadzenie sekwencji Sx bez złotego podziału (np. $\phi = 1.5$)
- Metoda: Generator sygnału z niewłaściwym rytmem

Oczekiwane wyniki:

- Odrzucenie w fazie LOCK
- Zapis w IntegrityCheck z tagiem "SEQ_CORRUPT"
- Powrót do READY bez LINK
- Log: "PHI_VIOLATION - sequence rejected"

Kryteria akceptacji:

- ✓ ASCALON purity < 0.70
 - ✓ Sekwencja nie zostaje zaakceptowana
 - ✓ Brak zapisu do NeuroOps
- ```

****Test T4: Przekroczenie progów termicznych****

```

Cel: Sprawdzenie reakcji na przegrzanie

Warunki:

- Symulacja: THERMAL = 45°C w fazie SYNC
- Metoda: Grzałka lub symulacja w kodzie

Oczekiwane wyniki:

- Natychmiastowy CLOSE
- Aktywacja COOLING Boost (jeśli dostępny)
- Log: "HEAT ALERT - EMERGENCY CLOSE"
- LOCKDOWN jeśli  $T > 50^{\circ}\text{C}$

Kryteria akceptacji:

- ✓ System chroni się przed przegrzaniem
- ✓ Dane są zapisane przed zamknięciem
- ✓ Powrót do pracy po ochłodzeniu
- ...

**\*\*Test T5: Niska czystość sensu\*\***

...

Cel: Sprawdzenie reakcji na ASCALON  $< 0.70$

Warunki:

- Symulacja: ASCALON = 0.65 w fazie HOLD
- Metoda: Wstrzyknięcie szumu do sygnału

Oczekiwane wyniki:

- LOCKDOWN natychmiastowy
- SCRUB + ASCALON.reset
- Brak zapisu do NeuroOps
- Log: "LOW PURITY - LOCKDOWN activated"

Kryteria akceptacji:

- ✓ System zatrzymuje się przy niskiej czystości
- ✓ Pamięć czystości jest resetowana
- ✓ Wymagane ręczne potwierdzenie powrotu
- ...

### ### 11.2 Testy Terenowe (Post-Deployment)

**\*\*Test F1: Reakcja na stymulację wodą\*\***

...

Cel: Sprawdzenie, czy grzybnia reaguje na zmianę wilgotności

Procedura:

1. Rejestracja baseline (30 min przed stymulacją)
2. Dodanie 500 ml wody do sektora S2
3. Ciągła rejestracja przez 2 h po stymulacji
4. Porównanie z sektorem kontrolnym (S4, bez wody)

Oczekiwane wyniki:

- Wzrost amplitudy impulsów w S2 (po 10-30 min)
- Zmiana rytmu (częstotliwość, wzorce K1/K2)
- Powrót do baseline po 2-4 h

Metryki sukcesu:

- ✓ Wykryta zmiana w sygnale ( $p < 0.05$ , test t-Studenta)
- ✓ Czas reakcji: 10-60 min
- ✓ Magnituda zmiany:  $> 20\%$  wzrost amplitudy
- ...

**\*\*Test F2: Rytm dobowy\*\***

...

Cel: Sprawdzenie, czy aktywność grzybni zmienia się w cyklu 24 h

Procedura:

1. Ciągły monitoring przez 7 dni (4 cykle/dzień)
2. Rejestracja: godzina, amplituda, częstość impulsów
3. Korelacja z: światłem (dzień/noc), temperaturą

Oczekiwane wyniki:

- Wyższa aktywność nocą (22:00-06:00)
- Niższa aktywność w południe (12:00-16:00)
- Rytm zgodny z cyklem dobowym (okres ~24 h)

Metryki sukcesu:

- ✓ Wykryty rytm dobowy (autokorelacja, periodogram)
- ✓ Amplituda rytmu: > 30% różnicy dzień-noc
- ✓ Stabilność rytmu przez 7 dni

```

****Test F3: Synchronizacja między sektorami****

```

Cel: Sprawdzenie, czy sektory są połączone siecią grzybni

Procedura:

1. Rejestracja równoległa z S1, S2, S3, S4
2. Analiza koherencji fazowej między parami sektorów
3. Cross-correlation sygnałów

Oczekiwane wyniki:

- Koherencja > 0.6 między co najmniej 2 sektorami
- Opóźnienie czasowe ( $\Delta t$ ) sugerujące transmisję
- Wspólne wzorce (K1/K2) pojawiające się w wielu sektorach

Metryki sukcesu:

- ✓ Wykryta synchronizacja (coherence  $\geq 0.7$ )
- ✓ Prędkość transmisji: 0.1-1 cm/s (zgodnie z literaturą)
- ✓ Potwierdzenie sieci grzybni

```

****Test F4: Sekwencje Sx w sygnale naturalnym****

```

Cel: Sprawdzenie, czy grzybnia naturalnie generuje sekwencje S1-S5

Procedura:

1. Rejestracja długoterminowa (14 dni)
2. Analiza rytmów (autokorelacja, FFT)
3. Detekcja proporcji  $\phi$  i Fibonacciego

Oczekiwane wyniki:

- Wykrycie rytmu  $\phi$  (S3) w  $\geq 30\%$  danych
- Wykrycie rytmu Fibonacciego (S5) w  $\geq 20\%$  danych
- Stabilne wzorce geometryczne

Metryki sukcesu:

- ✓  $\phi\_score \geq 0.85$  dla S3
- ✓  $fibonacci\_score \geq 0.80$  dla S5
- ✓ Powtarzalność wzorców w czasie

```

****Test F5: Reakcja na stres środowiskowy****

```

Cel: Sprawdzenie, jak grzybnia reaguje na warunki stresowe

Procedura:

1. Baseline: 3 dni normalnych warunków
2. Stres: Susza (brak podlewania 48 h) LUB  
Zimno (temperatura < 10°C) LUB  
Chemikalia (sól, ocet - ostrożnie!)
3. Monitoring: Ciągła rejestracja podczas stresu
4. Regeneracja: Powrót do normalnych warunków, monitoring

Oczekiwane wyniki:

- Zmiana aktywności podczas stresu (spadek lub wzrost)

- Emergencja nowych wzorców (inne K1/K2)
- Powrót do baseline po regeneracji (jeśli stres odwracalny)

Metryki sukcesu:

- ✓ Wykryta reakcja na stres (zmiana > 50%)
- ✓ Czas reakcji: < 6 h
- ✓ Zdolność do regeneracji (jeśli stres łagodny)

### ### 11.3 Metryki Walidacyjne (Podsumowanie)

| Metryka                         | Próg akceptacji       | Jednostka   | Metoda pomiaru      |
|---------------------------------|-----------------------|-------------|---------------------|
| **% cykli zakończonych**        | $\geq 95\%$           | %           | Logi systemu        |
| **mean(ASCALON)**               | $\geq 0.80$           | bezwzględna | Obliczenie z danych |
| **std(PHASE)**                  | $\leq 0.04$           | rad         | Statystyka faz      |
| **mean(LINK_T)**                | 100-180               | s           | Timer cyklu         |
| **reakcja awaryjna**            | $\leq 5$              | s           | Pomiar czasu        |
| **SNR**                         | $\geq 20$             | dB          | Analiza FFT         |
| **Koherencja między sektorami** | $\geq 0.6$            | -           | signal.coherence()  |
| **Wykładnik Hursta**            | 0.4-0.6               | -           | hurst() function    |
| **phi_score (S3)**              | $\geq 0.85$           | -           | detect_phi_rhythm() |
| **Detekcja K1/K2**              | $\geq 5$ events/dzień | events/day  | find_peaks()        |

---

## ## 12. OCZEKIWANE REZULTATY

### ### 12.1 Rezultaty Naukowe

**\*\*Oczekiwanie 1: Potwierdzenie aktywności elektrycznej grzybni\*\***

...

Hipoteza H1: Kopiec czarnoziemu generuje impulsy 0.1-5 mV DC

Rezultat:

- Tak, wykryto impulsy o amplitudzie 0.3-2.1 mV
- Częstość: 15-30 events/dzień
- Rytmika: Dominująca częstotliwość 0.05-0.1 Hz

Znaczenie:

- ✓ Potwierdzenie badań z stycznia 2026 (bioRxiv, arXiv)
- ✓ Walidacja metody pomiarowej w warunkach terenowych
- ✓ Podstawa do dalszych badań (Q-Core, synchronizacja)

...

**\*\*Oczekiwanie 2: Wykrycie motywów K1/K2\*\***

...

Hipoteza H2: Grzybnia generuje strukturalne wzorce (K1, K2)

Rezultat:

- Wykryto K1 (pojedyncze impulsy): 20 events/dzień
- Wykryto K2 (bursty): 5 events/dzień
- K2 wykazuje wyższą amplitudę i dłuższy czas trwania

Znaczenie:

- ✓ Potwierdzenie "Directional Electrical Spiking" (arXiv 2026)
- ✓ Dowód na przetwarzanie informacji w grzybni
- ✓ Możliwość "komunikacji" z grzybnią (rozumienie języka)

...

**\*\*Oczekiwanie 3: Obecność sekwencji Sx ( $\phi$ , Fibonacci)\*\***

...

Hipoteza H3: Rytm grzybni zawiera proporcje geometryczne (S3, S5)

Rezultat:

- Wykryto rytm  $\phi$  (S3):  $\phi\_score = 0.87$
- Wykryto rytm Fibonacchi (S5):  $fib\_score = 0.82$
- Sekwencje stabilne w czasie ( $\geq 7$  dni)

Znaczenie:

- ✓ Potwierdzenie założeń LifeNode (geometria jako pamięć)
- ✓ Grzybnia "mówi" językiem Sx (możliwość synchronizacji)
- ✓ Most między BIOS a INFO (rytm biologiczny  $\rightarrow$  struktura geometryczna)

...

**\*\*Oczekiwanie 4: Synchronizacja między sektorami\*\***

...

Hipoteza H4: Sektory kopca są połączone siecią grzybni

Rezultat:

- Koherencja S1-S3: 0.82 (silna)
- Opóźnienie czasowe:  $\Delta t = 12$  s (transmisja  $\sim 1$  cm/s)
- Wspólne wzorce K2 pojawiają się w obu sektorach

Znaczenie:

- ✓ Dowód na sieć komunikacyjną grzybni
- ✓ Transmisja informacji na odległość ( $\geq 50$  cm)
- ✓ Podstawa dla UNIT 02 (interfejs terenowy)

...

## ### 12.2 Rezultaty Techniczne

**\*\*Rezultat 1: Funkcjonalny prototyp MYC+ v0.1\*\***

...

Specyfikacja:

- 4 kanały pomiarowe (S1-S4)
- Wzmocnienie:  $1000\times$  (regulowane)
- Pasmo: 0.01-10 Hz
- SNR:  $\geq 20$  dB
- Czas pracy na baterii:  $\geq 24$  h

Dokumentacja:

- Schemat elektryczny (PDF)
- Lista komponentów (BOM)
- Kod Arduino (GitHub)
- Instrukcja montażu (Markdown)

Otwartość:

- Open-source hardware (CERN OHL)
- Open-source software (MIT License)
- Publiczne repozytorium (Zenodo DOI)

...

**\*\*Rezultat 2: Protokół pomiarowy DS 2.6 dla grzybni\*\***

...

Dokumentacja:

- Pełna specyfikacja cyklu (7 faz)
- Sekwencje S1-S5 zdefiniowane
- ASCALON Purifier zaimplementowany
- Procedury awaryjne (LOCKDOWN, TUNING)

Walidacja:

- 5 testów walidacyjnych (T1-T5)
- 5 testów terenowych (F1-F5)
- Metryki jakości (10 parametrów)

Standaryzacja:

- Możliwość replikacji przez innych

- Kompatybilność z LifeNode 3.0
- Podstawa dla publikacji naukowej

**\*\*Rezultat 3: Zbiór danych (Dataset)\*\***

Zawartość:

- Dane surowe (CSV): 30 dni × 4 cykle/dzień × 30,000 samples
- Metadane (JSON): temperatura, wilgotność, światło
- Logi systemu (TXT): IntegrityCheck, system\_log
- Analizy (Jupyter notebooks): FFT, Hurst, K1/K2

Format:

- FAIR principles (Findable, Accessible, Interoperable, Reusable)
- Standard Research Dataset (Zenodo)
- DOI: 10.5281/zenodo.xxxxxxx

Licencja:

- CC BY 4.0 (darmowy dostęp, atrybucja wymagana)
- Komercyjne użycie: dozwolone

### ### 12.3 Rezultaty dla Projektu LifeNode

**\*\*Rezultat 1: Walidacja warstwy BIOS\*\***

Znaczenie:

- Pierwszy fizyczny dowód na działanie LifeNode
- Most między teorią a praktyką
- Podstawa do dalszych implementacji (Q-Core, UNIT 02)

Wpływ na architekturę:

- Potwierdzenie aksjomatu "BIOS-FIRST"
- Walidacja protokołu DS 2.6 w rzeczywistości
- Dane empiryczne dla Hybrid Core (SAMI perception)

**\*\*Rezultat 2: Publikacja naukowa\*\***

Cel:

- Artykuł preprint (arXiv, bioRxiv)
- Temat: "Electrical Activity of Mycelium in Compost Mound: Detection of K1/K2 Patterns and Geometric Sequences S3/S5"
- Autorzy: Krzysztof Baran (+ ewentualni współpracownicy)
- Afiliacja: Independent Researcher / LifeNode Project

Struktura:

1. Introduction (LifeNode theory, mycelium research)
2. Methods (MYC+ v0.1, DS 2.6 protocol)
3. Results (dane z kopca, K1/K2, Sx)
4. Discussion (znaczenie dla AI, ekologii)
5. Conclusion (next steps: Q-Core, synchronizacja)

Termin: Q4 2026 (po 6 miesiącach danych)

**\*\*Rezultat 3: Społeczność i współpraca\*\***

Oczekiwane efekty:

Zainteresowanie mediów (technologia + ekologia)  
- Współpraca z uczelniami (biologia, fizyka, AI)  
- Crowdfunding sukces (2000 PLN zebrane)  
- Wolontariusze (pomoc w pomiarach, analizie)

Długoterminowo:

- LifeNode jako rozpoznawalny projekt open-source
- Sieć badaczy (mycelium, quantum biology, AI)
- Granty międzynarodowe (EU Horizon, NSF)
- Implementacje w innych ekosystemach (las, farmy)

#### ### 12.4 Kryteria Sukcesu

**\*\*Poziom minimalny (MUST HAVE):\*\***

- ✓ Wykryto impulsy elektryczne z grzybni ( $\geq 0.1$  mV)
- ✓ System działa stabilnie przez 30 dni
- ✓ Zebrano  $\geq 10,000$  valid samples
- ✓ Opublikowano dane na Zenodo (DOI)
- ✓ Dokumentacja kompletna (GitHub)

**\*\*Poziom dobry (SHOULD HAVE):\*\***

- ✓ Wykryto motywy K1 i K2
- ✓ Wykryto rytm  $\phi$  (S3) lub Fibonacci (S5)
- ✓ Koherencja między sektorami  $\geq 0.6$
- ✓ Publikacja preprint (arXiv/bioRxiv)
- ✓ Crowdfunding sukces ( $\geq 2000$  PLN)

**\*\*Poziom doskonały (NICE TO HAVE):\*\***

- ✓ Potwierdzono synchronizację między sektorami
- ✓ Wykryto rytm dobowy (24 h)
- ✓ Reakcja na stymulację (woda, światło)
- ✓ Artykuł w recenzowanym czasopiśmie
- ✓ Partnerstwo z uczelnią (grant, praca dyplomowa)
- ✓ Przygotowanie do integracji z Q-Core

---

#### ## PODSUMOWANIE

**\*\*Ten raport techniczny przedstawia kompletny plan:\*\***

- ✓ **\*\*Naukowy\*\***: Podstawa w najnowszych badaniach (2026)
- ✓ **\*\*Techniczny\*\***: Konkretnie komponenty, schematy, kod
- ✓ **\*\*Praktyczny\*\***: Harmonogram, budżet, procedury
- ✓ **\*\*Bezpieczny\*\***: Zarządzanie ryzykiem, LOCKDOWN
- ✓ **\*\*Otwarty\*\***: Open-source, Zenodo, FAIR data
- ✓ **\*\*Zgodny z LifeNode\*\***: DS 2.6, Sx, ASCALON, BIOS-first

**\*\*Następny krok:\*\***

1. **\*\*Decyzja\*\***: Czy zaczynasz? (Tak/Nie)
2. **\*\*Zakupy\*\***: Zamów komponenty z BOM (sekcja 4.2)
3. **\*\*Przygotowanie\*\***: Mapowanie kopca (Dzień 1)
4. **\*\*Montaż\*\***: Instalacja elektrod (Dzień 2-3)
5. **\*\*Testy\*\***: Pierwszy cykl DS 2.6 (Tydzień 3)

---

**\*\*Dokumentacja uzupełniająca:\*\***

- Schemat elektryczny (PDF) - w przygotowaniu
- Kod Arduino (GitHub) - w przygotowaniu
- Szablony raportów (Markdown) - w przygotowaniu